UNIT-III
Classification, Model Evaluation, Hyperparameter Tuning, and Feature Engineering

Rajesh Kumar (Assistant Prof.)

September 9, 2025

Abstract

This project presents a comprehensive machine learning workflow focusing on classification problems. The report covers fundamental classification algorithms, evaluation metrics, cross-validation techniques, hyperparameter tuning methods, and feature engineering approaches. A practical implementation using the Titanic dataset demonstrates the application of these concepts, with detailed analysis of model performance and feature importance.

Contents

1 Introduction **4**

2 Theoretical Background **4**
2.1 Classification Algorithms 4
2.1.1 Problem of Linear Regression for Classification 4
2.1.2 Sigmoid Function 5
2.1.3 Logistic Regression 6
2.2 Model Evaluation Metrics 6

3 Confusion Matrix **7**
3.1 Structure of a Confusion Matrix 7
3.1.1 True Positives (TP) 7
3.1.2 True Negatives (TN) 7
3.1.3 False Positives (FP) 7
3.1.4 False Negatives (FN) 7
3.2 Performance Metrics Derived from Confusion Matrix 7
3.3 Visual Representation of a Confusion Matrix 8
3.3.1 Performance Metrics 8

4 ROC-AUC Curve **9**
4.0.1 Key Definitions 10
4.0.2 Confusion Matrix and Derived Metrics 10
4.0.3 Sensitivity / True Positive Rate / Recall 11
4.1 False Negative Rate 11
4.2 Specificity / True Negative Rate 11
4.3 False Positive Rate 11
4.3.1 ROC Curve Visualization 12
4.3.2 Interpreting AUC Values 12
4.3.3 Conclusion 12

5 Model Evaluation Techniques **13**
5.0.1 Cross-Validation 13

6 Hyperparameter Tuning **15**
6.1 Techniques for Hyperparameter Tuning 15
6.2 GridSearchCV 15
6.3 RandomizedSearchCV 17
6.3.1 How RandomizedSearchCV Works 17

6.3.2 Advantages over GridSearchCV . . . 17
6.3.3 Example: Tuning Decision Tree with RandomizedSearchCV . . . 17
6.3.4 Parameter Distributions Explained . . . 18
6.3.5 Key RandomizedSearchCV Parameters . . . 18
6.3.6 Expected Output . . . 18

7 Feature Engineering **19**
7.0.1 Feature Creation . . . 19
7.0.2 Feature Transformation . . . 19
7.0.3 Feature Selection . . . 19
7.0.4 Handling Missing Values . . . 19
7.1 Feature Scaling . . . 20
7.1.1 Normalization . . . 20
7.1.2 Standardization . . . 20
7.2 Regularization Techniques . . . 21
7.2.1 L1 Regularization (Lasso) . . . 21
7.2.2 L2 Regularization (Ridge) . . . 21
7.3 Summary . . . 21

8 Experimental Setup **22**
8.1 Dataset Description . . . 22
8.2 Methodology . . . 22

1 Introduction

Machine learning classification represents one of the most important applications of artificial intelligence in various domains. This project explores the complete pipeline from data preparation to model deployment, emphasizing the critical aspects of model evaluation, hyperparameter optimization, and feature engineering that distinguish successful machine learning implementations.

2 Theoretical Background

2.1 Classification Algorithms

Classification in machine learning is a type of supervised learning approach where the goal is to predict the category or class of an instance that are based on its features. In classification it involves training model ona dataset that have instances or observations that are already labeled with Classes and then using that model to classify new, and unseen instances into one of the predefined categories.

Classification is a supervised learning technique where the goal is to predict categorical class labels based on input features. Unlike regression which predicts continuous values, classification deals with discrete outcomes.

2.1.1 Problem of Linear Regression for Classification

Linear regression is ill-suited for classification problems because:

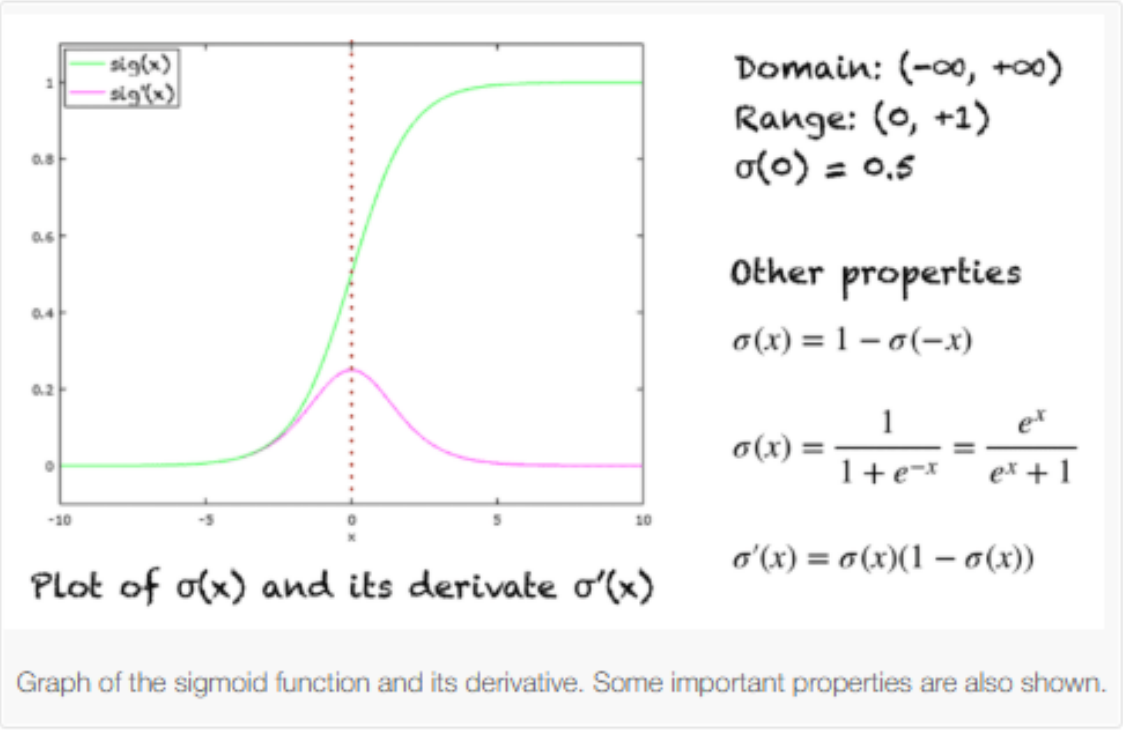
- It predicts continuous values while classification requires discrete categories
- Predictions can fall outside the [0,1] range for probability estimation
- It assumes a linear relationship between features and the target variable

2.1.2 Sigmoid Function

The sigmoid function transforms any real-valued number into a value between 0 and 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{1}$$

Where $z = w^T x + b$ is the linear combination of features and weights.



Graph of the sigmoid function and its derivative. Some important properties are also shown.

A few other properties of the sigmoid function $\sigma(z)$ include:

- **Domain:** $(-\infty, +\infty)$
- **Range:** $(0, 1)$
- $\sigma(0) = 0.5$
- The function is **monotonically increasing**.
- The function is **continuous everywhere**.
- The function is **differentiable everywhere** in its domain.

Numerically, it is sufficient to compute the function's value over a small range of numbers, e.g., $[-10, +10]$. For values $z < -10$, the function's value is asymptotically close to zero ($\sigma(z) \approx 0$). For values $z > 10$, the function's value is asymptotically close to one ($\sigma(z) \approx 1$).

2.1.3 Logistic Regression

In Logistic regression is classification algorithm used to estimate discrete values, typically binary, such as 0 and 1, yes or no. It predicts the probability of an instance belonging to a class that makes it essectial for binary classification problems like spam detection or diagnosing disease.
Logistic functions are ideal for classification problems since their output is between 0 and 1. Many fields employ it because of its simplicity, interpretability, and efficiency. Logistic Regression works well when features and event probability are linear. Logistic Regression used for binary classification tasks. Logistic regression is used for binary categorization. Despite its name, it predicts class membership likelihood. A logistic function models probability in this linear model.
Logistic regression models the probability that a given input belongs to a particular category:

$$P(y = 1|x) = \frac{1}{1 + e^{-(w^T x+b)}} \tag{2}$$

The cost function for logistic regression is:

$$J(w) = -\frac{1}{m}\sum_{i=1}^{m}[y^{(i)} \log(h(x^{(i)})) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))] \tag{3}$$

2.2 Model Evaluation Metrics

Model evaluation is a process that uses some metrics which help us to analyze the performance of the model. Think of training a model like teaching a student. Model evaluation is like giving them a test to see if they truly learned the subject—or just memorized the answers. It helps us answer critical questions:

- Did the model learn general patterns?
- Will it fail on new, unseen questions?

Model development is a multi-step process, and we need to keep a check on how well the model makes future predictions and analyze a model's weaknesses. There are many metrics for that purpose.

Cross-Validation is one technique that is followed during the training phase, and it is a fundamental model evaluation technique.

3 Confusion Matrix

A confusion matrix is a performance evaluation tool used in machine learning that summarizes the performance of a classification model by tabulating true positive, true negative, false positive, and false negative predictions. It helps assess the accuracy and effectiveness of the model's predictions.

The matrix is based on the concepts of:

- **True Positives (TP)**: Correct positive predictions
- **True Negatives (TN)**: Correct negative predictions
- **False Positives (FP)**: Incorrect positive predictions
- **False Negatives (FN)**: Incorrect negative predictions

3.1 Structure of a Confusion Matrix

3.1.1 True Positives (TP)

Instances where the model correctly predicts a positive class when it is indeed positive. Consider a cancer diagnostic model: a true positive would occur when the model correctly identifies a patient with cancer as having the disease. TP is a vital measure of the model's ability to recognize positive instances accurately.

3.1.2 True Negatives (TN)

Instances where the model correctly predicts a negative class when it is indeed negative. Continuing the medical analogy, a true negative would be when the model correctly identifies a healthy patient as not having the disease. TN reflects the model's proficiency in recognizing negative instances.

3.1.3 False Positives (FP)

Instances where the model incorrectly predicts a positive class when it should have been negative. In the medical scenario, a false positive would mean the model wrongly indicates a patient has the disease when they are, in fact, healthy. FP illustrates instances where the model exhibits overconfidence in predicting positive outcomes.

3.1.4 False Negatives (FN)

Instances where the model incorrectly predicts a negative class when it should have been positive. In the medical context, a false negative would be when the model fails to detect a disease in a patient who actually has it. FN highlights situations where the model fails to capture actual positive instances.

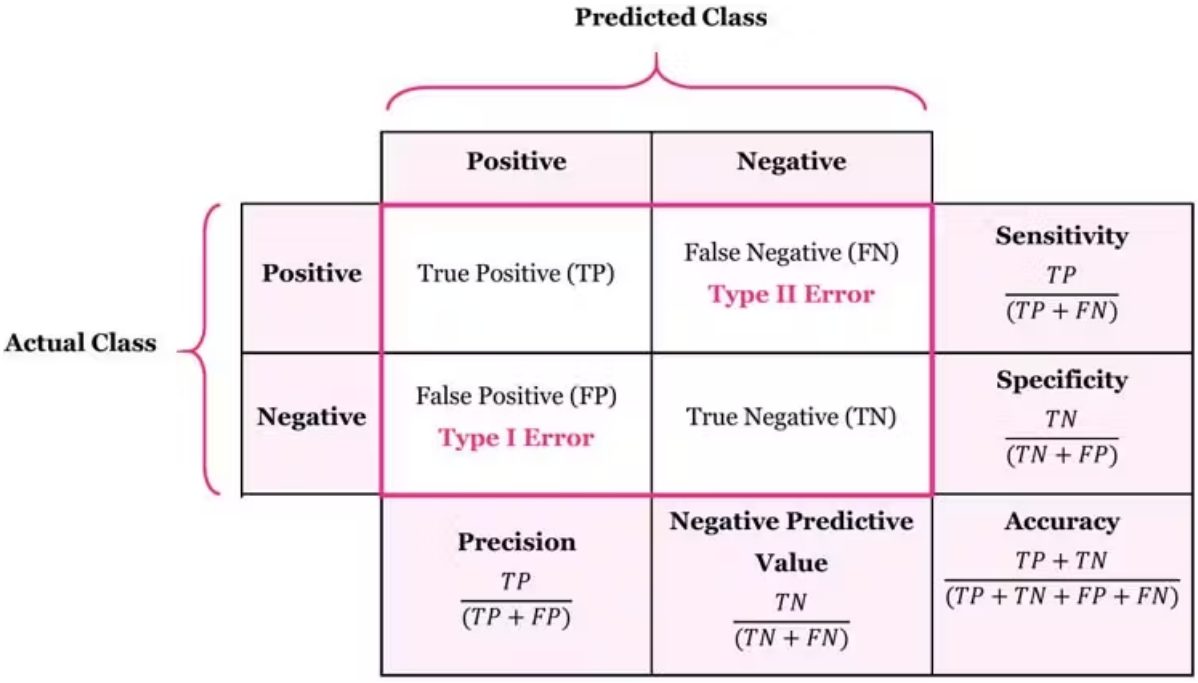
3.2 Performance Metrics Derived from Confusion Matrix

The confusion matrix serves as the basis for calculating essential evaluation metrics:

- **Accuracy**: $\frac{TP+TN}{TP+TN+FP+FN}$ - Ratio of correct predictions to total predictions

- **Precision**: $\frac{TP}{TP+FP}$ - Proportion of true positives among positive predictions
- **Recall/Sensitivity**: $\frac{TP}{TP+FN}$ - Ratio of true positives to actual positive instances
- **Specificity**: $\frac{TN}{TN+FP}$ - Ratio of true negatives to actual negative instances
- **F1-Score**: $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$ - Harmonic mean of precision and recall

3.3 Visual Representation of a Confusion Matrix

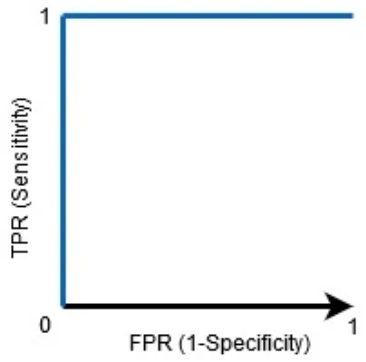


3.3.1 Performance Metrics

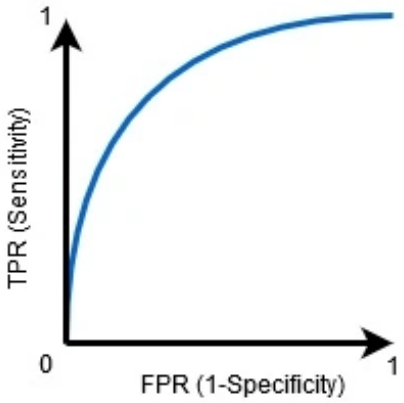
- **Accuracy**: $\frac{TP+TN}{TP+TN+FP+FN}$
- **Precision**: $\frac{TP}{TP+FP}$
- **Recall/Sensitivity**: $\frac{TP}{TP+FN}$
- **F1-score**: $2 \times \frac{Precision \times Recall}{Precision+Recall}$
- **Specificity**: $\frac{TN}{TN+FP}$

4 ROC-AUC Curve

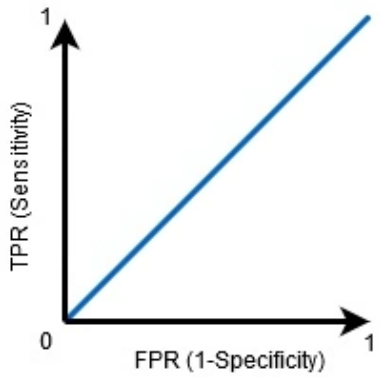
An ROC curve, or receiver operating characteristic curve, is like a graph that shows how well a classification model performs. It helps us see how the model makes decisions at different levels of certainty. The curve has two lines: one for how often the model correctly identifies positive cases (true positives) and another for how often it mistakenly identifies negative cases as positive (false positives). By looking at this graph, we can understand how good the model is and choose the threshold that gives us the right balance between correct and incorrect predictions.
The Receiver Operator Characteristic (ROC) curve is an evaluation metric for binary classification problems. It is a probability curve that plots the TPR against FPR at various threshold values and essentially separates the 'signal' from the 'noise.' In other words, it shows the performance of a classification model at all classification thresholds. The Area Under the Curve (AUC) is the measure of the ability of a binary classifier to distinguish between classes and is used as a summary of the ROC curve.



When AUC = 1, the classifier can correctly distinguish between all the Positive and the Negative class points. If, however, the AUC had been 0, then the classifier would predict all Negatives as Positives and all Positives as Negatives.



When $0.5 < \text{AUC} < 1$, there is a high chance that the classifier will be able to distinguish the positive class values from the negative ones. This is so because the classifier is able to detect more numbers of True positives and True negatives than False negatives and False positives.



The ROC (Receiver Operating Characteristic) curve is a graphical representation that shows how good a model is at distinguishing between classes. It helps us visualize how often the model correctly identifies positive instances and how often it correctly avoids labeling negative instances as positive. Essentially, it demonstrates the performance of a binary classification model across all classification thresholds.

4.0.1 Key Definitions

- **AUC (Area Under the Curve)**: A single metric representing the overall performance of a binary classification model based on the area under its ROC curve.
- **ROC Curve (Receiver Operating Characteristic Curve)**: A graphical plot illustrating the trade-off between True Positive Rate and False Positive Rate at various classification thresholds.
- **True Positive Rate (Sensitivity/Recall)**: Proportion of actual positives correctly identified by the model.

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- **False Positive Rate**: Proportion of actual negatives incorrectly classified as positives by the model.

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

- **Specificity (True Negative Rate)**: Proportion of actual negatives correctly identified by the model.

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}} = 1 - \text{FPR}$$

4.0.2 Confusion Matrix and Derived Metrics

A confusion matrix provides the foundation for calculating these important metrics:

Table 1: Confusion Matrix Structure

	Actual Values		
	Positive	**Negative**	
Predicted Positive	True Positive (TP)	False Positive (FP)	
Predicted Negative	False Negative (FN)	True Negative (TN)	

4.0.3 Sensitivity / True Positive Rate / Recall

Sensitivity tells us what proportion of the positive class got correctly classified.

$$\text{Sensitivity} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

A simple example would be determining what proportion of the actual sick people were correctly detected by the model.

4.1 False Negative Rate

False Negative Rate (FNR) tells us what proportion of the positive class got incorrectly classified by the classifier.

$$\text{FNR} = \frac{\text{FN}}{\text{TP} + \text{FN}} = 1 - \text{Sensitivity}$$

A higher TPR and a lower FNR are desirable since we want to classify the positive class correctly.

4.2 Specificity / True Negative Rate

Specificity tells us what proportion of the negative class got correctly classified.

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

Taking the same example as in Sensitivity, Specificity would mean determining the proportion of healthy people who were correctly identified by the model.

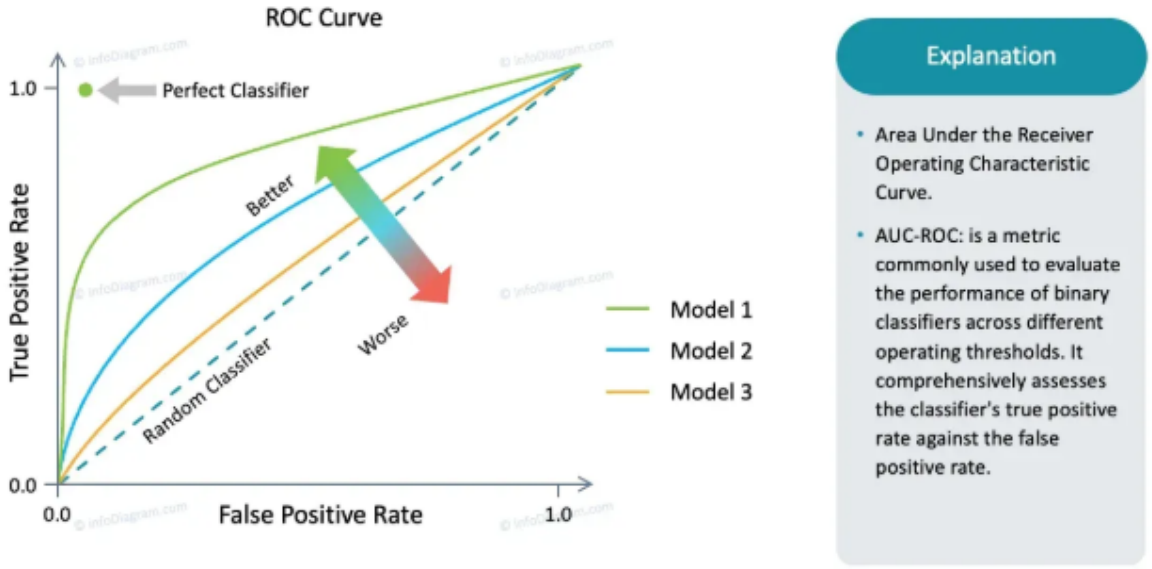
4.3 False Positive Rate

FPR tells us what proportion of the negative class got incorrectly classified by the classifier.

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} = 1 - \text{Specificity}$$

A higher TNR and a lower FPR are desirable since we want to classify the negative class correctly.

4.3.1 ROC Curve Visualization



4.3.2 Interpreting AUC Values

The Area Under the Curve (AUC) provides a single measure of classifier performance:

- $\mathbf{AUC = 1.0}$: Perfect classifier - achieves 100% sensitivity and 100% specificity
- $0.5 < \mathbf{AUC} < 1$: Better than random guessing - the classifier has some discrimination ability
- $\mathbf{AUC = 0.5}$: No discrimination ability - equivalent to random guessing
- $AUC < 0.5$: Worse than random guessing - indicates something is wrong with the model

4.3.3 Conclusion

The ROC curve and AUC provide comprehensive tools for evaluating binary classification models. By visualizing the trade-off between true positive rate and false positive rate across all possible thresholds, these metrics offer a more nuanced understanding of model performance than simple accuracy measures. Sensitivity and specificity are fundamental concepts that form the basis of these evaluation techniques, allowing data scientists to select appropriate thresholds based on the specific requirements of their application.

5 Model Evaluation Techniques

5.0.1 Cross-Validation

Cross Validation is a method in which we do not use the whole dataset for training. In this technique some part of the dataset is reserved for testing the model. There are many types of Cross-Validation out of which K Fold Cross Validation is mostly used. In K Fold Cross Validation the original dataset is divided into k subsets. The subsets are known as folds. This is repeated k times where 1 fold is used for testing purposes, rest k-1 folds are used for training the model. It is seen that this technique generalizes the model well and reduces the error rate.

Holdout is the simplest approach. It is used in neural networks as well as in many classifiers. In this technique the dataset is divided into train and test datasets. The dataset is usually divided into ratios like 70:30 or 80:20. Normally a large percentage of data is used for training the model and a small portion of dataset is used for testing the model.

- **K-fold Cross-Validation**: Data divided into K subsets, each used as test set once
- **Leave-One-Out Cross-Validation**: Extreme case where K equals number of samples

K-fold cross validation in machine learning cross-validation is a powerful technique for evaluating predictive models in data science. It involves splitting the dataset into k subsets or folds, where each fold is used as the validation set in turn while the remaining k-1 folds are used for training. This process is repeated k times, and performance metrics such as accuracy, precision, and recall are computed for each fold. By averaging these metrics, we obtain an estimate of the model's generalization performance. This method is essential for model assessment, selection, and hyperparameter tuning, offering a reliable measure of a model's effectiveness. Compared to leave-one-out cross-validation, which uses k equal to the number of samples, K-fold cross-validation is computationally efficient and widely used in practice.

In each set (fold) training and the test would be performed precisely once during this entire process. It helps us to avoid overfitting. As we know when a model is trained using all of the data in a single short and give the best performance accuracy. To resist this k fold cross validation in machine learning cross-validation helps us to build the model is a generalized one.

To achieve this K-Fold Cross Validation, we have to split the data set into three sets, Training, Testing, and Validation, with the challenge of the volume of the data.

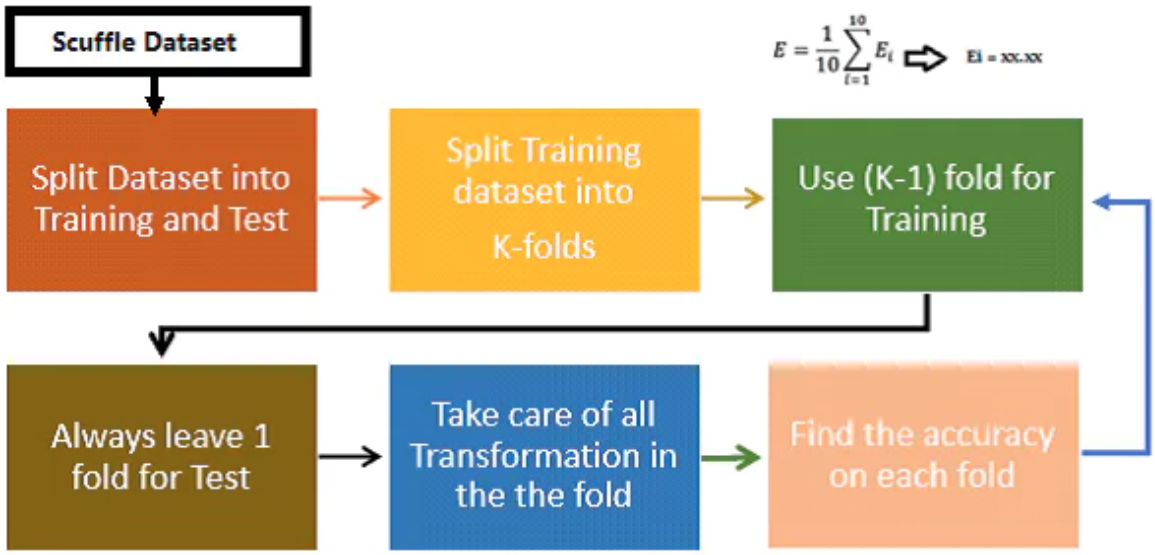
Also, you can checkout this about top 7 cross validation techniques with Python Code

Here Test and Train data set will support building model and hyperparameter assessments.

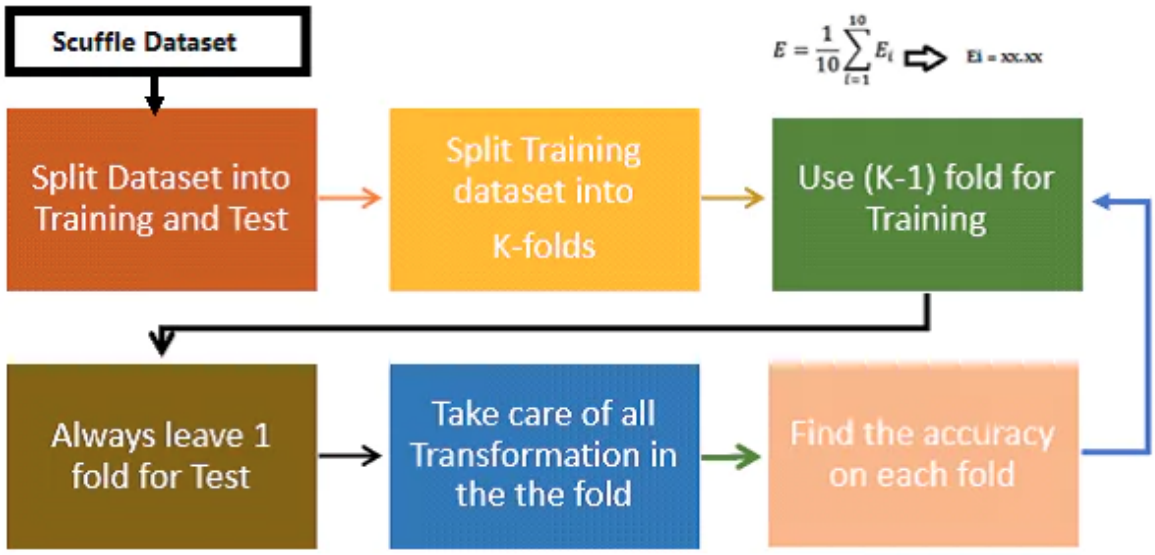
In which the model has been validated multiple times based on the value assigned as a parameter and which is called K and it should be an INTEGER.

Make it simple, based on the K value, the data set would be divided, and train/testing will be conducted in a sequence way equal to K time.

Life Cycle of K-Fold Cross-Validation



Let's have a generalised K value. If K=5, it means, in the given dataset and we are splitting into 5 folds and running the Train and Test. During each run, one fold is considered for testing and the rest will be for training and moving on with iterations, the below pictorial representation would give you an idea of the flow of the fold-defined size.



6 Hyperparameter Tuning

Hyperparameter tuning is the process of selecting the optimal values for a machine learning model's hyperparameters. These are typically set before the actual training process begins and control aspects of the learning process itself. They influence the model's performance its complexity and how fast it learns.
For example the learning rate and number of neurons in a neural network in a neural network or the kernel size in a support vector machine can significantly impact how well the model trains and generalizes. The goal of hyperparameter tuning is to find the values that lead to the best performance on a given task.
These settings can affect both the speed and quality of the model's performance.

- **A high learning rate** can cause the model to converge too quickly possibly skipping over the optimal solution.
- **A low learning rate** might lead to slower convergence and require more time and computational resources.

6.1 Techniques for Hyperparameter Tuning

Models can have many hyperparameters and finding the best combination of parameters can be treated as a search problem. The two best strategies for Hyperparameter tuning are:

- **Grid Search**: Exhaustive search over specified parameter values
- **Random Search**: Random selection of parameter combinations from specified distributions

6.2 GridSearchCV

GridSearchCV is a brute-force technique for hyperparameter tuning. It trains the model using all possible combinations of specified hyperparameter values to find the best-performing setup. It is slow and uses a lot of computer power which makes it hard to use with big datasets or many settings. It works using below steps:

- Create a grid of potential values for each hyperparameter.
- Train the model for every combination in the grid.
- Evaluate each model using cross-validation.
- Select the combination that gives the highest score.

For example if we want to tune two hyperparameters C and Alpha for a Logistic Regression Classifier model with the following sets of values:
C = [0.1, 0.2, 0.3, 0.4, 0.5]
Alpha = [0.01, 0.1, 0.5, 1.0]

C \ Alpha	0.1	0.2	0.3	0.4
0.5	0.701	0.703	0.697	0.696
0.4	0.699	0.702	0.698	0.702
0.3	0.721	0.726	0.713	0.703
0.2	0.706	0.705	0.704	0.701
0.1	0.698	0.692	0.688	0.675

The goal is to find the best combination of two hyperparameters, C and α (regularization strength), for a Logistic Regression model. We do this by training and evaluating the model on every single possible combination of the values we provide.

- **C (Inverse of Regularization Strength):** A lower value of C (e.g., 0.1) applies **stronger regularization**, which helps prevent overfitting but might underfit the data. A higher value of C (e.g., 0.5) applies **weaker regularization**, allowing the model to fit the training data more closely but risking overfitting.
- **Alpha (Regularization Strength):** This is the direct regularization term. A higher value of α (e.g., 1.0) applies **stronger regularization**. A lower value (e.g., 0.01) applies **weaker regularization**.

Important Note: It's unusual to tune both C and α simultaneously, as they are inversely related ($C = 1/\alpha$). Typically, you would tune only one of them. This grid might be for a specific library implementation or a pedagogical example.

The provided sets of values create a grid of $5 \times 4 = 20$ unique combinations to test.

- **C values:** $[0.1, 0.2, 0.3, 0.4, 0.5]$ (listed vertically on the left)
- **Alpha values:** $[0.01, 0.1, 0.5, 1.0]$ (listed horizontally at the top)

Each cell in the grid represents the result of training a model with a specific (C, α) pair and evaluating it on a validation set (likely using a metric like **accuracy** or **F1-score**).

How to read a cell: The value in each cell is the model's performance score for that hyperparameter combination.

Example: The cell where $C = 0.3$ and $\alpha = 0.1$ has a score of 0.726. This means the model with those parameters achieved a performance score of 72.6%.

6.3 RandomizedSearchCV

Key Concept As the name suggests, **RandomizedSearchCV** picks **random combinations** of hyperparameters from the given ranges instead of checking every single combination like GridSearchCV.

6.3.1 How RandomizedSearchCV Works

The algorithm follows this iterative process:

1. In each iteration, it tries a new **random combination** of hyperparameter values
2. It records the model's performance for each combination using cross-validation
3. After several attempts (specified by the user), it selects the **best-performing** set of hyperparameters

6.3.2 Advantages over GridSearchCV

- **Computationally efficient** - doesn't evaluate all combinations
- **Faster convergence** to good hyperparameters
- **Better for large parameter spaces** where exhaustive search is impractical
- Can discover **unexpected good combinations** that might be missed in a grid

6.3.3 Example: Tuning Decision Tree with RandomizedSearchCV

The following code illustrates how to use `RandomizedSearchCV`:

```
import numpy as np
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=1000, n_features=20, n_informative=10, n_classes=2, random_state=42)

from scipy.stats import randint
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    "max_depth": [3, None],
    "max_features": randint(1, 9),
    "min_samples_leaf": randint(1, 9),
    "criterion": ["gini", "entropy"]
}

tree = DecisionTreeClassifier()
tree_cv = RandomizedSearchCV(tree, param_dist, cv=5)
tree_cv.fit(X, y)

print("Tuned Decision Tree Parameters: {}".format(tree_cv.best_params_))
print("Best score is {}".format(tree_cv.best_score_))
```

Table 2: Hyperparameter Ranges for Decision Tree

Parameter	Type	Description
`max_depth`	Integer	Maximum depth of the tree (1-20)
`min_samples_split`	Integer	Minimum samples required to split a node (2-10)
`min_samples_leaf`	Integer	Minimum samples required at a leaf node (1-10)
`criterion`	Categorical	Function to measure split quality
`max_features`	Categorical	Number of features to consider for best split

6.3.4 Parameter Distributions Explained

6.3.5 Key RandomizedSearchCV Parameters

- `n_iter`: Number of parameter settings that are sampled (default: 10)
- `param_distributions`: Dictionary with parameters names as keys and distributions to sample from
- `cv`: Cross-validation splitting strategy (e.g., 5 for 5-fold CV)
- `scoring`: Strategy to evaluate the performance of the cross-validated model
- `random_state`: Seed for random number generation for reproducibility

6.3.6 Expected Output

The code will output something like:

```
Best parameters: {'criterion': 'gini', 'max_depth': 8,
                  'max_features': 'sqrt', 'min_samples_leaf': 2,
                  'min_samples_split': 5}
Best cross-validation score: 0.9667
```

Performance Note RandomizedSearchCV typically finds **90-95%** of the optimal performance of GridSearchCV while only evaluating **5-10%** of the parameter combinations, making it much more computationally efficient.

7 Feature Engineering

Feature engineering is the process of transforming raw data into features that are suitable for use by machine learning algorithms, thereby improving the performance and accuracy of predictive models. It involves using domain knowledge and various data manipulation techniques to create new features or modify existing ones. Feature engineering plays a critical role in the performance of machine learning models. Key aspects include:

7.0.1 Feature Creation

Generating new features from existing ones to capture more relevant information. This can involve:

- Combining features,
- Extracting components (e.g., day of the week from a date),
- Creating interaction terms.

7.0.2 Feature Transformation

Modifying the scale or distribution of features to meet the requirements of certain algorithms or improve model stability. Examples include:

- Scaling (normalization, standardization),
- Encoding categorical variables (one-hot encoding, label encoding),
- Handling outliers.

7.0.3 Feature Selection

Identifying and selecting the most relevant features for the model while discarding irrelevant or redundant ones. This can:

- Reduce dimensionality,
- Improve model interpretability,
- Prevent overfitting.

7.0.4 Handling Missing Values

Addressing missing data points through techniques such as:

- Imputation (replacing missing values with estimated ones),
- Deletion of missing values.

The goal of feature engineering is to enhance the ability of machine learning algorithms to learn patterns from the data and make more accurate predictions. It is a crucial step in the machine learning workflow and can significantly impact the success of a model.

7.1 Feature Scaling

Feature scaling is a crucial preprocessing step in machine learning that ensures the features are on a similar scale. Two common methods for feature scaling are normalization and standardization.

7.1.1 Normalization

Normalization refers to the process of scaling data to fit within a specified range, typically $[0, 1]$. This method is particularly useful for algorithms that are sensitive to the scale of data, such as gradient descent-based methods.

The formula for normalization is:

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{4}$$

Where:

- x is the original feature value.
- $x_{\min}$ is the minimum value in the feature.
- $x_{\max}$ is the maximum value in the feature.

Advantages of Normalization:

- It ensures that all features contribute equally to the distance measurement.
- Helpful when features have different units (e.g., height in cm vs. weight in kg).

7.1.2 Standardization

Standardization transforms the data to have a mean of 0 and a standard deviation of 1. This method is often useful when the features have different distributions.

The formula for standardization is:

$$x_{\text{std}} = \frac{x - \mu}{\sigma} \tag{5}$$

Where:

- x is the original feature value.
- μ is the mean of the feature.
- σ is the standard deviation of the feature.

Advantages of Standardization:

- It is particularly effective for algorithms that assume a Gaussian distribution of the data.
- Helps accelerate convergence when using gradient descent.

7.2 Regularization Techniques

Regularization is an essential method used to prevent overfitting in machine learning models. By adding a penalty term to the cost function, it discourages complex models and encourages simpler, more generalizable solutions.

7.2.1 L1 Regularization (Lasso)

Lasso (Least Absolute Shrinkage and Selection Operator) adds the absolute value of the coefficients as a penalty term to the cost function. The cost function with L1 regularization is formulated as:

$$J(w) = \text{MSE} + \lambda \sum_{i=1}^{n} |w_i| \tag{6}$$

Where:

- $J(w)$ is the cost function.
- MSE is the mean squared error.
- λ is the regularization parameter that controls the strength of the penalty.
- w_i are the coefficients (weights) assigned to the features.

Advantages of L1 Regularization:

- Can lead to sparse solutions where some weights are exactly zero, effectively performing feature selection.
- Helps improve model interpretability by reducing the number of features.

7.2.2 L2 Regularization (Ridge)

Ridge regression adds the squared value of the coefficients as a penalty term to the cost function. The cost function with L2 regularization is defined as:

$$J(w) = \text{MSE} + \lambda \sum_{i=1}^{n} w_i^2 \tag{7}$$

Where the terms are similar to those defined in L1 regularization.
Advantages of L2 Regularization:

- Does not necessarily lead to sparse solutions; instead, it shrinks all coefficients, leading to a more balanced model.
- Helps to avoid multicollinearity, improving model performance where features are correlated.

7.3 Summary

Feature scaling ensures that all features contribute equally during the learning process by normalizing or standardizing their values. Regularization techniques, such as L1 and L2, introduce penalties to the model's complexity, preventing overfitting and enhancing generalization. Both aspects are foundational in building robust machine learning models.

8 Experimental Setup

8.1 Dataset Description

The Titanic dataset contains information about passengers aboard the RMS Titanic, with the target variable being survival (0 = Died, 1 = Survived). The dataset includes features such as passenger class, age, sex, number of siblings/spouses aboard, number of parents/children aboard, fare, and port of embarkation.

8.2 Methodology

The project follows this workflow:

1. Data loading and exploratory analysis
2. Feature engineering and preprocessing
3. Model selection and training
4. Hyperparameter tuning
5. Model evaluation and comparison
6. Feature importance analysis